Fundamentals of System Security
(COMSM0122)

Workshop 3: Understanding and Stress-Testing Paxos

Prerequisite:
(1) A code editor such as Visual Studio Code
(2) Node.js (version 18 or newer)
(3) Basic familiarity with JavaScript
(4) A GitHub account
(5) Completed the lecture on Paxos

Workshop Learning Objectives
This workshop will provide you with practical experience of the Paxos consensus algorithm by allowing you to observe, modify, and experiment with a simplified JavaScript implementation of Paxos. Through a series of guided exercises, you will explore how distributed systems preserve a single agreed decision despite crashes, node failures, network partitions, competing proposers, and unreliable communication. You will investigate the roles of proposers, acceptors, learners, quorums, proposal numbers, and majority agreement, and develop an understanding of why Paxos preserves consistency by relying on overlapping quorums and protocol rules rather than any individual node.

The workshop also develops the developer mental models introduced in the lecture. You will experiment with scenarios involving acceptor failures, competing proposals, partial acceptance, previously chosen values, Multi-Paxos optimisation, network partitions, state loss, quorum misconfiguration, and deployment mistakes to understand the assumptions upon which Paxos guarantees depend. By the end of the workshop, you should be able to reason about the distinction between protocol correctness and operational reliability, identify common failure modes in Paxos-based systems, and critically evaluate how violations of protocol assumptions can undermine consensus guarantees even when the algorithm itself is correctly implemented.

You are encouraged to work in teams of 2-3 students.

Setup
In this workshop, you will work with a simplified JavaScript-based Paxos simulator designed specifically for learning and experimentation. The simulator models a distributed system containing proposers, acceptors, learners, quorums, proposal numbers, and message exchanges. Unlike a production Paxos implementation, the simulator has been intentionally simplified to make the core concepts of majority agreement, value preservation, proposal ordering, and fault tolerance easier to understand and explore.

The simulator contains several predefined scenarios that allow you to observe how Paxos behaves under both normal operating conditions and a variety of failure scenarios. You will progressively explore node failures, competing proposers, partial acceptance, preservation of previously chosen values, Multi-Paxos optimisation,

network partitions, state loss, quorum misconfiguration, and deployment mistakes. Throughout the workshop, you are encouraged to modify the code, inspect protocol state, analyse message exchanges, and evaluate protocol outcomes to understand not only how consensus is achieved, but also how and why it can fail when its assumptions are violated.

Directory Structure

```
paxos-lab/
├── package.json
├── simulator.js
├── proposal.js
├── proposer.js
├── acceptor.js
├── learner.js
├── quorum.js
├── network.js
└── scenarios/
    ├── happy_path.js
    ├── acceptor_failure.js
    ├── competing_proposers.js
    ├── partial_acceptance.js
    ├── preserve_chosen_value.js
    ├── multi_paxos.js
    ├── network_partition.js
    ├── breaking_assumptions/
    │   ├── majority_loss.js
    │   ├── state_loss.js
    │   └── quorum_misconfiguration.js
    └── hardening.js
```

Component Responsibilities

(1) Proposal (`proposal.js`)
Represents a Paxos proposal.
Each proposal contains:
- A proposal number
- A proposed value

Proposal numbers provide ordering and allow Paxos to resolve contention between competing proposers.
This component supports the mental model: **Proposal numbers create order.**

(2) Proposer (`proposer.js`)
Represents a participant responsible for initiating consensus.
Responsibilities:
- Create proposals
- Generate increasing proposal numbers
- Initiate the Prepare phase

- Coordinate agreement on a value

You will later observe multiple proposers competing for control of the consensus process.

(3) Acceptor (`acceptor.js`)
Represents the most important role in Paxos.
Responsibilities:
- Receive Prepare requests
- Issue Promises
- Accept proposals
- Remember previously accepted values
- Participate in quorum formation

Acceptors collectively determine which values can be chosen.
This component supports the mental model: **A majority is the system's memory.**

(4) Learner (`learner.js`)
Represents components that learn the final decision.
Responsibilities:
- Receive chosen values
- Update local state
- Record agreed outcomes

Learners do not vote and do not influence the consensus process.
This component reinforces the mental model: **Learners learn decisions rather than make them.**

(5) Quorum Engine (`quorum.js`)
Determines the minimum number of acceptors required for agreement.
Responsibilities:
- Calculate majority size
- Verify quorum formation
- Enforce quorum-based safety

For a three-node deployment: `Acceptors = 3` and `Quorum = 2`
You will later modify quorum calculations to observe how incorrect quorum rules can violate Paxos safety guarantees.
This component reinforces: **Quorums preserve history through overlap.**

(6) Network Simulator (`network.js`)
Provides a simplified representation of communication between participants.
Responsibilities:
- Record message exchanges
- Simulate message delivery
- Simulate message loss
- Model network partitions

Network-related exercises will allow you to explore:

- Communication failures
- Network partitions
- Message loss
- Availability limitations

This supports the mental model: **Paxos assumes failures will occur.**

(7) Consensus Engine (`simulator.js`)
Implements the core Paxos protocol.
Responsibilities:
- Generate proposals
- Collect promises
- Collect acceptances
- Validate quorum formation
- Preserve previously accepted values
- Notify learners

This component embodies the rule: **Protect agreement even when progress becomes difficult.**

(8) Failure and Recovery Scenarios
Unlike the Byzantine Fault Tolerance workshop, Paxos does not focus on malicious behaviour. Instead, the workshop explores failures that arise when the assumptions of the protocol are violated.
These failures are implemented directly within the simulator components and scenario files rather than through a dedicated attack framework.

Examples include:

Failure	**Scenario**
Acceptor Failure	acceptor_failure.js
Network Partition	network_partition.js
Majority Loss	majority_loss.js
State Loss	state_loss.js
Quorum Misconfiguration	quorum_misconfiguration.js
Proposer Competition	competing_proposers.js

These exercises allow students to explore how Paxos responds when nodes become unavailable, communication is disrupted, state is lost, or protocol assumptions are weakened.
This reinforces the workshop mental model: **Paxos assumes participants are honest, but not always available.**

(9) Scenario Files
Each workshop exercise is provided as a self-contained scenario.

Scenario	**Purpose**
`happy_path.js`	Normal consensus execution
`acceptor_failure.js`	Consensus despite node failure
`competing_proposers.js`	Resolving proposal conflicts

`partial_acceptance.js`	Preserving partially accepted values
`preserve_chosen_value.js`	Preserving chosen values
`multi_paxos.js`	Stable proposer emergence
`network_partition.js`	Communication failure analysis
`majority_loss.js`	Loss of quorum
`state_loss.js`	Loss of persistent state
`quorum_misconfiguration.js`	Safety violation through incorrect quorum mathematics
`hardening.js`	Defensive engineering and mitigation

Design Philosophy

This simulator is intentionally educational rather than production-grade. The architecture has been designed to make protocol assumptions visible, encourage experimentation, and allow you to directly observe the consequences of failures and operational mistakes.

Throughout the workshop, you should view the simulator not simply as a consensus implementation, but as a laboratory for exploring agreement, fault tolerance, quorum formation, value preservation, and the assumptions that underpin Paxos.

A central theme throughout the workshop is: Paxos does not ask "What if someone lies?"; Paxos asks "What if honest participants fail?"

Understanding that distinction is essential for understanding both Paxos and the broader family of distributed consensus protocols.

Installation

Download the workshop repository from Blackboard and install the required dependencies by running:

```
npm install
```

Running a Scenario

To execute the default scenario:

```
npm start
```

You can also run individual exercises directly:

```
npm run happy
npm run failure
npm run proposers
npm run partial
npm run preserve
npm run multi
npm run partition
npm run majority-loss
npm run state-loss
npm run quorum-error
npm run hardening
```

Workshop Approach
The workshop follows an experimentation-based approach. Rather than implementing Paxos from scratch, your task is to observe protocol behaviour, introduce failures, and reason about the resulting outcomes.
For each task, consider the following questions:
- What assumptions does the protocol make?
- Which nodes are required for progress?
- What information must be preserved?
- What happens when nodes become unavailable?
- What happens when communication is disrupted?
- Does Paxos preserve safety?
- Does Paxos preserve liveness?
- Which mental model from the lecture best explains the observed behaviour?

As you progress through the tasks, you should begin to recognise that Paxos is fundamentally about preserving agreement despite failures, using quorums, proposal ordering, and value preservation rather than relying on any individual participant.

Part 1: Understanding Consensus
The purpose of this part is to introduce the core Paxos workflow and establish the most important mindset in Paxos: **Preserve agreement, even when participants fail.**

You begin by observing a consensus round in which all participants behave correctly and no failures occur. The goal is to understand how a proposer coordinates a decision, how acceptors form a majority, and how learners discover the final result. You should become familiar with proposal numbers, promises, acceptance, learning, and quorum formation before introducing failures.

Tasks
1. Run the `happy_path.js` scenario.
2. Observe the Prepare, Promise, Accept, and Learn phases.
3. Identify the chosen value.
4. Record the quorum size.
5. In maximum of 200 words, explain why consensus was reached.

Reflection Questions
- Which role initiated the consensus round?
- At what point was the value considered chosen?
- What role did the acceptors play?
- What role did the learners play?
- Why was a majority required?

You should understand that Paxos achieves consensus through majority agreement rather than relying on a central coordinator.

Part 2: Acceptor Failure
The purpose of this part is to demonstrate that Paxos tolerates failures by requiring a majority rather than unanimity.

You will simulate the failure of an acceptor and observe how consensus can still be achieved despite one participant becoming unavailable.

Tasks

1. Run the `acceptor_failure.js` scenario.
2. Observe the failed acceptor.
3. Compare the output with the happy-path scenario.
4. Identify the chosen value.
5. In maximum of 200 words, explain why consensus still succeeded.

Reflection Questions

- Why did consensus continue despite a failure?
- How many acceptors can fail in this deployment?
- What assumption is Paxos making?
- Would consensus still succeed if another acceptor failed?

You should understand that Paxos tolerates minority failures because decisions depend on majorities, not unanimous agreement.

Part 3: Competing Proposers

The purpose of this part is to demonstrate how Paxos resolves competition between multiple proposers.

You will introduce competing proposers and observe how proposal numbers establish order and prevent conflicting decisions.

Tasks

1. Run the `competing_proposers.js` scenario.
2. Observe the Prepare and Accept messages.
3. Identify which proposer succeeds.
4. Record which proposal number wins.
5. In maximum of 200 words, explain why one proposer was rejected.

Reflection Questions

- Why does the higher proposal number win?
- Why are proposal numbers necessary?
- Does a higher proposal number imply a better decision?
- What would happen if proposal numbers did not exist?

You should understand that proposal numbers do not determine truth. They determine which proposer is allowed to continue.

Part 4: Partial Acceptance

The purpose of this part is to explore one of the most important Paxos safety mechanisms.

You will observe a proposer that partially succeeds before another proposer takes over. The exercise illustrates how Paxos preserves previously accepted values and prevents conflicting outcomes.

Tasks

1. Run the `partial_acceptance.js` scenario.
2. Observe the value reported by the acceptor.
3. Observe the value originally proposed by the second proposer.
4. Identify the final chosen value.
5. In maximum of 200 words, explain why the value changed.

Reflection Questions

- Why was the second proposer forced to adopt the earlier value?
- What information must acceptors remember?
- What would happen if acceptors forgot their history?
- How does this preserve safety?

You should understand that proposers may change, but previously accepted values must survive.

Part 5: Preserving Chosen Values

The purpose of this part is to demonstrate how Paxos protects decisions that may already have been chosen.

You will observe a new proposer attempting to introduce a new value after a previous value has already been accepted by a majority.

Tasks

1. Run the `preserve_chosen_value.js` scenario.
2. Observe the value that was previously chosen.
3. Observe the value desired by the new proposer.
4. Identify the final chosen value.
5. In maximum of 200 words, explain why the original value remained unchanged.

Reflection Questions

- Why can future proposers not overwrite previously chosen values?
- What role does quorum overlap play?
- Which Paxos safety property is being preserved?
- What would happen if previously chosen values could be replaced?

You should understand that Paxos protects decisions, not proposers.

Part 6: Multi-Paxos and Stable Proposers

The purpose of this part is to explore how Paxos can become more efficient through the emergence of a stable proposer.

You will observe repeated consensus rounds coordinated by the same proposer and investigate the intuition behind Multi-Paxos.

Tasks

1. Run the `multi_paxos.js` scenario.
2. Observe the sequence of proposal numbers.

3. Identify the proposer coordinating each round.
4. Compare the rounds and outcomes.
5. In maximum of 200 words, explain how stability emerges.

Reflection Questions

- Does Paxos explicitly elect a leader?
- Why does one proposer eventually dominate?
- How does this improve efficiency?
- What problem does Multi-Paxos address?

You should understand that Paxos chooses values, while Multi-Paxos allows a stable proposer to emerge naturally.

Part 7: Breaking Paxos Assumptions

This part explores what happens when Paxos assumptions stop being true.

Unlike Byzantine Fault Tolerance, Paxos is primarily concerned with failures rather than malicious behaviour. The exercises in this section focus on infrastructure, deployment, and operational failures that can undermine progress or correctness.

Part 7.1: Network Partitioning

The purpose of this part is to investigate how communication failures affect consensus.

You will simulate a network partition and observe how message delivery can be disrupted.

Tasks

1. Run the `network_partition.js` scenario.
2. Observe which messages are delivered and which are dropped.
3. Compare the behaviour before and after the partition.
4. In maximum of 200 words, explain the impact of the partition.

Reflection Questions

- Which protocol property is most affected?
- Does a partition necessarily violate safety?
- Does a partition prevent progress?
- Why does Paxos prefer consistency over availability?

You should understand that network partitions primarily threaten progress rather than agreement.

Part 7.2: Majority Loss

The purpose of this part is to demonstrate the importance of quorum availability.

You will remove enough acceptors to prevent a majority from forming.

Tasks

1. Run the `majority_loss.js` scenario.
2. Observe quorum calculation.
3. Record how many acceptors remain available.
4. Explain why the protocol stops.

Reflection Questions

- Which requirement is no longer satisfied?
- Why can Paxos no longer proceed?
- Is safety violated?
- Is liveness violated?

You should understand that Paxos cannot make progress without a majority.

Part 7.3: State Loss

The purpose of this part is to demonstrate the importance of persistent storage.
You will simulate an acceptor losing its previously accepted state.
Tasks

1. Run the `state_loss.js` scenario.
2. Observe the state before and after failure.
3. Identify the information that was lost.
4. In maximum of 200 words, explain why this is dangerous.

Reflection Questions

- What information must acceptors remember?
- Why is persistent storage important?
- What could happen if multiple acceptors forget history?
- Which Paxos assumption has been violated?

You should understand that Paxos safety relies upon reliable persistence of protocol state.

Part 7.4: Quorum Misconfiguration

The purpose of this part is to investigate why quorum mathematics are security-critical.
You will weaken quorum requirements and observe the consequences.
Tasks

- Run the `quorum_misconfiguration.js` scenario.
- Examine the modified quorum rule.
- Observe the possible outcomes.
- Explain why the configuration is unsafe.

Reflection Questions

- What property has been lost?
- Why are overlapping quorums important?
- Could two conflicting values be chosen?
- What lesson does this teach about protocol configuration?

You should understand that quorum overlap is fundamental to Paxos safety.

Part 8: Hardening a Paxos Deployment

The purpose of this part is to move from failure analysis to system engineering.

In the previous exercises, you explored how failures, configuration problems, and operational mistakes can undermine Paxos assumptions. In this section, you will design and implement mitigations that strengthen the system against those failures.

You should recognise that Paxos remains correct only when its assumptions are adequately protected.

Scenario
You have inherited a Paxos implementation that has experienced:

- Loss of majority
- Acceptor state loss
- Network partitions
- Quorum misconfiguration
- Stable proposer instability

Your task is to modify the simulator and implement one or more defensive mechanisms.

A successful solution should:

- prevent failures,
- detect failures,
- reduce their impact,
- or improve recovery.

Task 8.1: Protect Quorum Rules
Current Weakness
A developer weakens the quorum rule and accidentally removes quorum overlap.

Hardening Strategy
Implement quorum validation `validateQuorum()` to reject unsafe quorum configurations before deployment.

On completion of this task, you will understand that protocol parameters are security-critical.

Task 8.2: Detect State Loss
Current Weakness
An acceptor loses previously accepted values after a restart.

Hardening Strategy
Implement state integrity validation to detect:

```
highestPromised = 0
acceptedProposal = null
```

when previously stored information should exist.

On completion of this task, you will understand that safety depends on durable storage.

Task 8.3: Protect Against Majority Loss

Current Weakness
The system continues accepting requests even when no majority exists.

Hardening Strategy
Implement health checks that disable new proposals when quorum cannot be formed.

On completion of this task, you will understand that consensus requires quorum availability.

Task 8.4: Detect Proposer Instability
Current Weakness
Multiple proposers continuously compete.

Hardening Strategy
Implement monitoring that detects excessive proposal churn.

On completion of this task, you will understand that a protocol can remain correct while becoming inefficient.

Task 8.5: Improve Observability
Current Weakness
The system provides little visibility into protocol decisions.

Hardening Strategy
Extend audit logs to record:

- Proposal numbers
- Accepted values
- Promise responses
- Quorum formation
- Chosen values

On completion of this task, you will understand that distributed systems must be observable as well as correct.

Part 9 (Optional Extension): Improving Paxos
This optional extension encourages you to think like a secure systems engineer. Rather than simply observing Paxos, design and implement improvements that strengthen the implementation.

Suggested Extensions

- Add persistent storage.
- Add state recovery.
- Add proposer-election heuristics.
- Add monitoring and alerting.
- Add failure detection.
- Add network diagnostics.
- Add protocol auditing.
- Design and implement your own enhancement.

Deliverables
For each enhancement:

1. Describe the failure being addressed.
2. Explain the assumption being protected.
3. Describe the implementation.
4. Demonstrate the enhancement.
5. Discuss remaining limitations.

Reflection Questions

- Which failures can be prevented?
- Which failures can only be detected?
- How does the enhancement affect complexity?
- Does the enhancement introduce new assumptions?
- What additional capabilities would a production Paxos deployment require?

At the end of this task, you should be able to critically evaluate the assumptions behind Paxos, identify operational risks, and propose practical engineering improvements that strengthen the reliability of distributed consensus systems.

Submission
Submit your implementation of both tasks as a zip file on Blackboard.
Notes:

1. Submission points will only be open on the day of the workshop.

2. You must be physically present in the lab to submit (except permitted otherwise)

3. You will not gain marks for submitting.

4. Submissions help us differentiate learning and engagement levels when marking both group and individual coursework.

   For all submissions before group assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.
   For all submissions after group assessment and before individual assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.

5. Finally, push your `paxos-lab` project folder to GitHub. Ensure that the project is associated with appropriate documentation and a README.md description. Tag your project appropriately, including the workshop date and name. Finally, invite the GitHub user systemsecuritybcsg to your pushed project. This GitHub handle is managed by the unit director.